Informe de Análisis Algorítmico y Resultados Experimentales

Proyecto: Sistema de Wave Picking para Warehouse (Multi-Picker por Zonas)

Materia: Algoritmos y Estructuras de Datos II

Repositorio Github: https://github.com/Markoss9/PROYECTO-WAVE

1. Introducción y Modelado del Problema

El presente informe detalla el diseño, implementación y análisis de un sistema automatizado para la preparación de pedidos mediante la estrategia de Wave Picking. El objetivo central radica en optimizar los recorridos de la flota de operarios (pickers) dentro del almacén, minimizando la distancia, respetando los límites de carga y reduciendo el costo computacional.

1.1 Representación del Warehouse y Estructuras de Datos

Para modelar el centro de distribución, el espacio se discretizó en una grilla bidimensional de coordenadas cartesianas (x,y). Sin embargo, para la ejecución eficiente de los algoritmos de recorrido y pathfinding, esta grilla fue abstraída y transformada dinámicamente en un Grafo Dirigido.

- **Nodos:** Representan las celdas transitables (pasillos) y las celdas de acceso perimetrales a los racks y zonas de despacho.
- **Aristas:** Representan la conectividad ortogonal (N, S, E, O). No se permiten movimientos diagonales.
- **Pesos y Direccionalidad:** Toda transición válida posee un costo unitario de distancia (1 paso). La direccionalidad del grafo está impuesta estrictamente por el sentido de circulación asignado a cada pasillo.

Justificación Estructural:

Se optó por representar este grafo subyacente utilizando una Lista de Adyacencia. En entornos de grilla ortogonal, cada nodo posee a lo sumo 4 vecinos. Esto define un grafo fuertemente disperso ($E << V^2$). Utilizar una Matriz de Adyacencia tradicional generaría un desperdicio de memoria inaceptable de $O(V^2)$ con valores nulos, mientras que la lista de adyacencia optimiza la retención en memoria a $O(V + E)$ y permite recuperar los sucesores de una coordenada en tiempo constante $O(1)$, lo cual es crítico para acelerar la propagación durante la Búsqueda en Anchura (BFS).

2. Decisiones Algorítmicas y Análisis de Complejidad

El núcleo del motor de ruteo aborda simultáneamente la consolidación de las waves, la división del trabajo (Order Splitting) y el cálculo de la ruta óptima intra-zona.

2.1 Algoritmo de Búsqueda de Rutas (BFS)

Dado que las distancias tienen costo unitario y no existen ponderaciones variables por tráfico dinámico en esta versión, el problema de encontrar el camino mínimo entre un picker y un producto se resuelve de manera teóricamente óptima utilizando Búsqueda en Anchura (BFS).

- **Complejidad Temporal:** O(V + E) en el peor caso, donde V es la cantidad total de celdas de pasillo habilitadas.
- **Complejidad Espacial:** O(V) necesaria para encolar la frontera de expansión y mantener el registro de nodos visitados.
- **Desempeño Práctico:** Extremadamente rápido para almacenes de dimensión estándar (como el layout 18x16 ensayado), logrando tiempos de milisegundos por ruta.

2.2 Consolidación y Estrategias de Ruteo

El sistema implementa una segmentación por zonas para evitar cruces innecesarios, dividiendo el trabajo de la Wave entre dos operarios (ej. PK_01 en Zona Izquierda y PK_02 en Zona Derecha). Una vez que el sistema Greedy asigna los productos, probamos tres enfoques para recorrerlos:

- **Estrategia A (Orden de los Pedidos):** El operario visita las ubicaciones respetando secuencialmente el orden en que ingresaron las líneas de pedido al sistema.
- **Estrategia B (Cercanía 3D - Greedy):** Tras recoger un ítem, el algoritmo BFS evalúa todos los destinos restantes en la lista del operario y se dirige al punto que reporte el menor costo de desplazamiento inmediato.
- **Estrategia C (Barrido de Pasillos):** Simula el comportamiento empírico de un humano. Ordena los productos según su coordenada de pasillo ascendente/descendente (eje X e Y) formando un flujo continuo tipo "serpiente", evitando el backtracking (retrocesos).

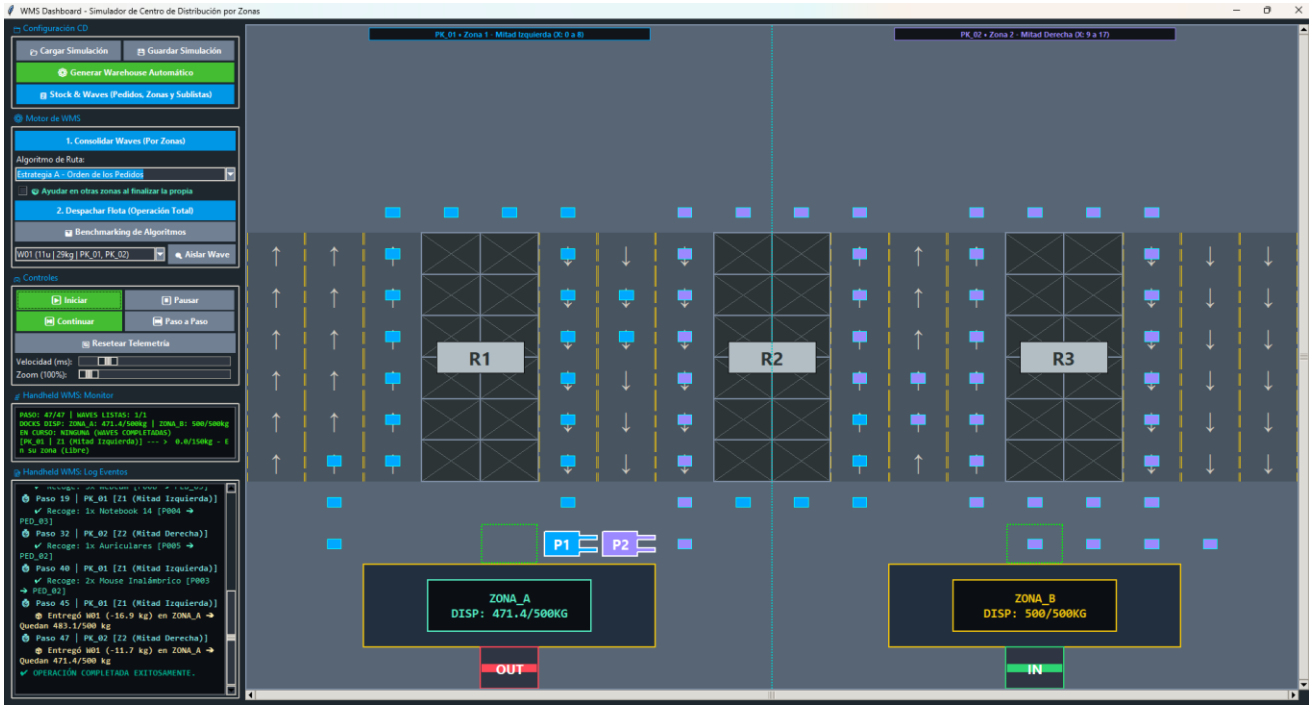
3. Resultados Experimentales

Para contrastar teóricamente estos enfoques, se ejecutó una simulación sobre un layout básico de 18x16 (dimensiones cartesianas), empleando 3 pasillos por zona (6 en total), procesando 1 Wave consolidada con una demanda de 11 unidades y 29kg de peso total, distribuida entre PK_01 y PK_02.

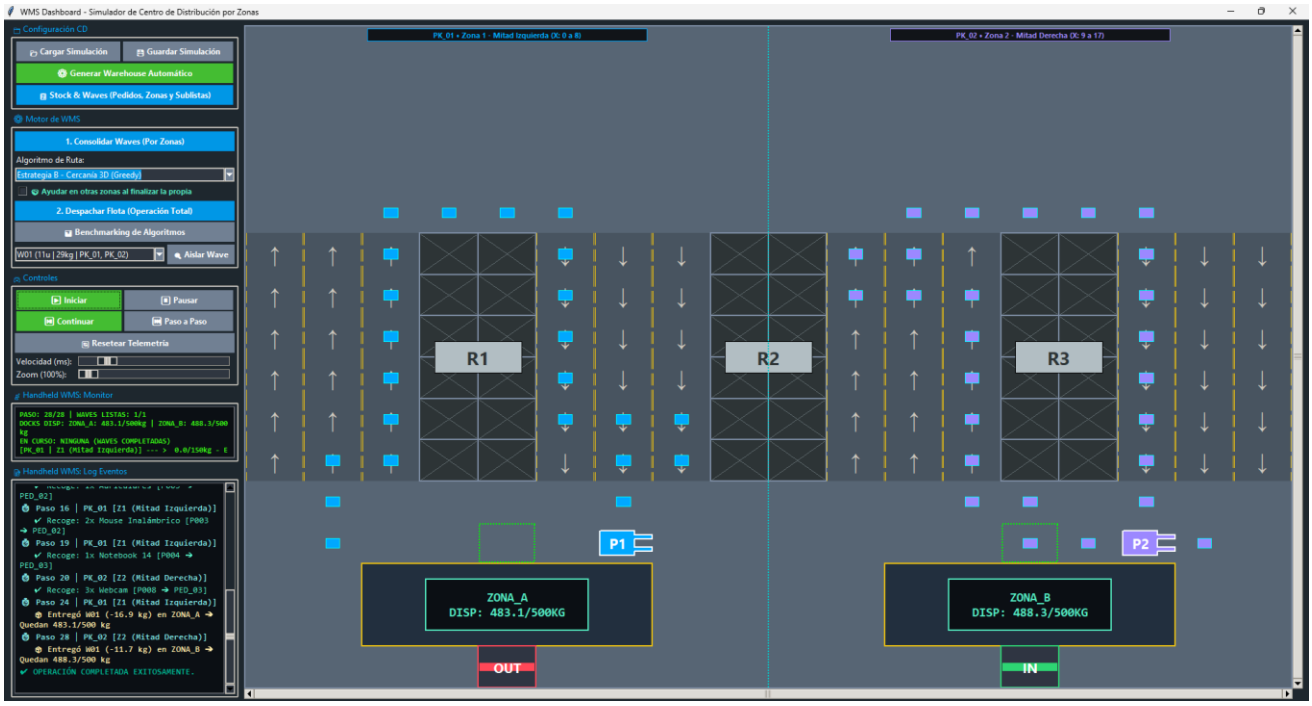
A continuación se presentan los registros telemétricos extraídos directamente del motor WMS:

Escenario Básico (18x16)	Estrategia de Ruteo	Distancia (Pasos Totales)	Nodos Visitados	Tiempo de CPU
Flota en paralelo	A - Orden de los Pedidos	92	71	11.69 ms
(Zona Izquierda + Derecha)	B - Cercanía 3D (Greedy)	52	51	0.47 ms
	C - Barrido de Pasillos	52	51	0.56 ms

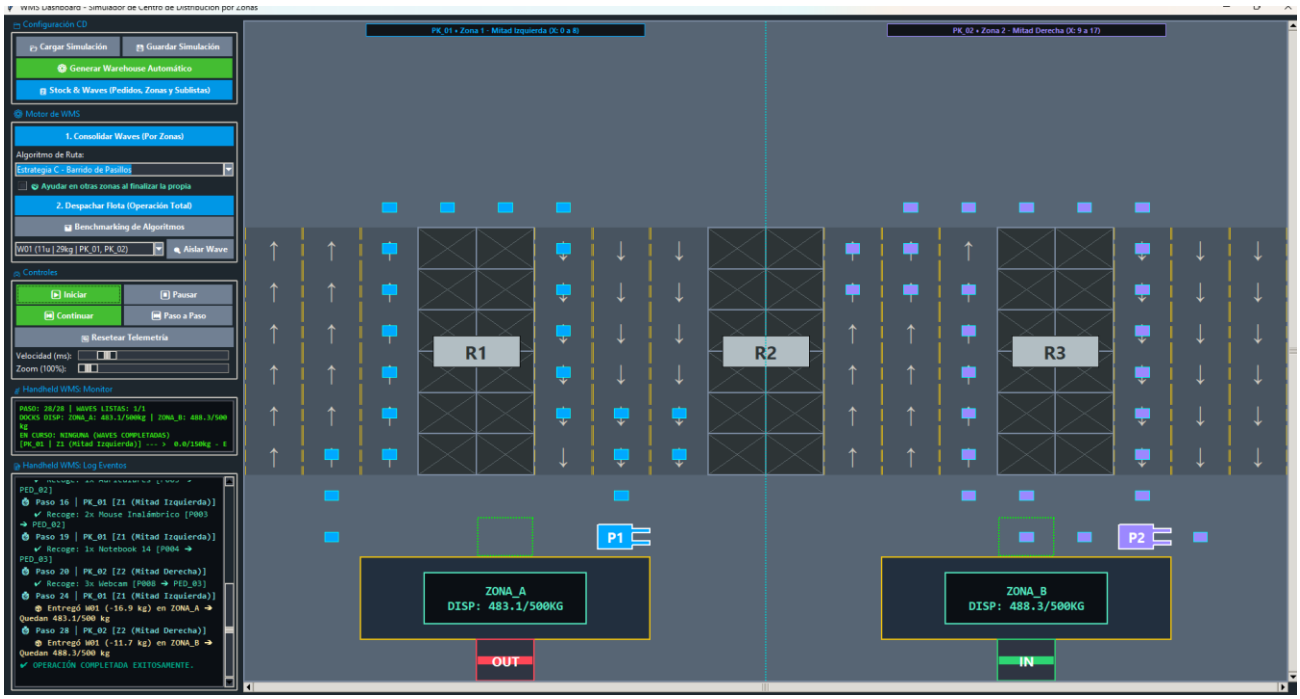
Visualización Operativa: Estrategia A (Orden de Pedidos)



Visualización Operativa: Estrategia B (Cercanía Greedy)



Visualización Operativa: Estrategia C (Barrido de Pasillos)



3.1 Análisis Analítico de las Diferencias

1. La ineficiencia del secuencialismo (Estrategia A): Como evidencia la tabla empírica, forzar a la flota a recolectar productos en el estricto orden administrativo del cliente provoca un efecto de "ping-pong". PK_01 y PK_02 deben atravesar el mismo pasillo múltiples veces, duplicando casi la distancia física (92 pasos vs 52). A nivel algorítmico, esto fuerza al BFS a calcular más rutas largas de extremo a extremo, reflejándose en un brutal aumento del tiempo de CPU (11.69 ms).

2. Eficiencia Heurística (Estrategia B y C): La técnica golosa por cercanía (B) y la de Barrido (C) demostraron ser inmensamente superiores en este escenario, empatando en el óptimo local de 52 pasos físicos y reduciendo los nodos procesados de 71 a 51.

3. Trade-offs de Tiempo y Distancia: Curiosamente, al resolver la permutación por Cercanía (B), el tiempo de CPU bajó a un mínimo histórico de 0.47 ms. Esto ocurre porque al visitar los vecinos más cercanos primero, los caminos que el BFS debe encontrar son muy cortos (la búsqueda "corta" de manera temprana en el árbol de exploración). En contraste con la Estrategia A, los 11 milisegundos se deben a que el BFS debió expandirse por casi toda la planta múltiples veces para conectar puntos distantes.

4. Zonificación: El diseño del simulador impide que PK_01 cruce a la zona de PK_02, garantizando que dos operarios operen en simultáneo sin colisiones centrales, dividiendo el problema principal en dos sub-grafos independientes.

4. Respuestas al Cuestionario Teórico

1. ¿Cómo fue modelado el Warehouse? (estructura de datos) El Warehouse fue modelado lógicamente como una grilla bidimensional cartesiana (matriz de celdas), donde cada elemento (racks, pasillos, zonas de despacho, puertas) ocupa coordenadas específicas (x, y). Sin embargo, para la ejecución del motor de ruteo, esta grilla se abstrajo en un **Grafo Dirigido**.

2. ¿Qué representación de la estructura de datos utilizaron y por qué? Para el grafo de ruteo, utilizamos una **Lista de Adyacencia** (implícita en la validación de vecinos). Se eligió esta representación porque una grilla ortogonal es un grafo sumamente disperso (E << V^2), ya que cada nodo tiene máximo 4 vecinos). Una matriz de adyacencia desperdiciaría una cantidad inaceptable de memoria O(V^2) con conexiones nulas, mientras que la lista de adyacencia optimiza el espacio a O(V+E) y permite recorrer los vecinos transitables en tiempo O(1).

3. ¿Qué algoritmos utilizaron para resolver los problemas encontrados?

- Para el *Pathfinding* (rutas): **Búsqueda en Anchura (BFS)**.
- Para la Consolidación de Waves y Ordenamiento de paradas: Algoritmos heurísticos tipo **Greedy** (golosos).

4. ¿Por qué eligieron dichos algoritmos? Se eligió BFS porque en una cuadrícula donde el costo de moverse de una celda a otra siempre es 1 (grafo no ponderado), BFS garantiza encontrar siempre el camino más corto de forma óptima y eficiente. Se eligió un enfoque Greedy para agrupar pedidos y ordenar las paradas porque calcular la permutación matemáticamente perfecta (Problema del Viajante o TSP) es NP-Hard; el enfoque Greedy entrega soluciones suficientemente buenas en un tiempo de cómputo casi instantáneo, ideal para un simulador en tiempo real.

5. ¿Qué ocurre si todos los pasillos tienen el mismo largo? Se genera una alta simetría en el grafo. Al haber múltiples caminos mínimos de la misma longitud exacta, el algoritmo BFS se ve obligado a explorar (encolar) una frontera de búsqueda mucho más amplia en todas direcciones antes de llegar al objetivo, aumentando la cantidad de nodos evaluados y el tiempo de CPU, aunque el costo final del camino sea el mismo.

6. ¿Cómo cambia el comportamiento del algoritmo cuando aumenta el tamaño del Warehouse? Al aumentar las dimensiones (ej. de 18x16 a 50x50), la cantidad de vértices (V) del grafo crece sustancialmente. Aunque la complejidad del BFS se mantiene lineal O(V+E), en términos absolutos el algoritmo debe procesar más celdas, lo que incrementa el tiempo de ejecución de manera proporcional a la cantidad de celdas transitables que separan el origen del destino.

7. ¿Cómo determinaron la distancia entre las ubicaciones de una wave? La distancia se midió contando la cantidad total de aristas transitadas (movimientos ortogonales efectivos de celda a

celda) en la ruta devuelta por el algoritmo BFS, respetando siempre el sentido de circulación obligatorio de cada pasillo (N, S, E, O).

8. ¿Cómo decidieron qué pedidos pertenecen a una misma wave? Utilizando la técnica de *Order Splitting*. Iteramos la lista de pedidos pendientes ordenados por prioridad y fuimos asignando sus productos a la Wave hasta alcanzar el límite paramétrico de unidades o el límite de peso soportado por el operario (peso_max_kg), asegurando no sobrecargar la zona de despacho (500kg).

9. ¿Qué estrategia utilizaron para ordenar las ubicaciones de una wave? Depende de la estrategia seleccionada en la UI:

- *Estrategia A:* Secuencial (respeta el orden de entrada del pedido).
- *Estrategia B:* Heurística del Vecino Más Cercano (Greedy). Desde el nodo actual, evalúa la distancia a los ítems restantes y se dirige al más próximo.
- *Estrategia C:* Heurística de Barrido. Ordena los productos espacialmente a lo largo de un eje (ej. por coordenada X creciente o Y creciente) para evitar retrocesos.

10. ¿Qué diferencias existen entre las estrategias implementadas? La Estrategia A es ciega al layout físico, priorizando la lógica administrativa del pedido, lo que causa un severo efecto "ping-pong". Las Estrategias B y C ignoran el orden del pedido y priorizan la ubicación geométrica de los productos para armar una ruta fluida.

11. ¿Cuál presenta menor distancia recorrida? Tanto la **Estrategia B (Cercanía/Greedy)** como la **Estrategia C (Barrido)** empataron presentando la menor distancia recorrida, logrando resolver el lote en solo **52 pasos**, frente a los 92 pasos que requirió la Estrategia A.

12. ¿Cuál presenta menor tiempo de ejecución? La **Estrategia B (Cercanía/Greedy)**, con un tiempo de **0.47 ms**. Al ir siempre al vecino más cercano, los caminos que el BFS debe encontrar son cortos. En contraste, la Estrategia A demoró **11.69 ms**, ya que obligó al BFS a calcular rutas largas de extremo a extremo de la planta repetidas veces.

13. ¿Existe una estrategia que sea mejor en todos los escenarios? No de forma absoluta. En almacenes pequeños, las tres estrategias pueden dar distancias similares. En almacenes masivos, la Estrategia B (Greedy) es universalmente superior a la A para acortar distancias, pero si hay una densidad extrema de pedidos, la Estrategia C (Barrido) suele ser la mejor en un entorno real porque evita que los pickers colisionen o generen tráfico pesado al ir y venir.

14. ¿Qué trade-offs encontraron entre tiempo de ejecución y calidad del recorrido? Paradójicamente, en nuestro simulador, forzar el orden administrativo (Estrategia A) no solo empeoró la calidad del recorrido (casi duplicó la distancia), sino que también empeoró el tiempo de ejecución de CPU al obligar a hacer búsquedas BFS más extensas. El trade-off real está en que la Estrategia B gasta un ínfimo tiempo extra en ordenar la lista de prioridades por cercanía

(ordenamiento O(N^2) por cada paso), pero se recupera inmediatamente al hacer invocaciones BFS más cortas y eficaces.

15. ¿Qué modificaciones realizarían para utilizar el sistema en un Warehouse real? El BFS asume que los pasillos siempre están libres (costo estático 1). En un Warehouse real con decenas de montacargas, hay "tráfico". Reemplazaríamos BFS por un algoritmo como **A*** o **Dijkstra** que permita asignar pesos dinámicos a las aristas (si un pasillo está congestionado, la arista "pesa" más). Además, implementaríamos cálculos de cubicaje (volumen de los productos) y no solo kilogramos para el límite de los vehículos.